Python Engineering

WEEK 03 - Modules & Coordinates | CLASS 12 - Engineering Toolkit | EX20

Objetivo: construir un pequeno modulo reutilizable de algebra vectorial para ingenieria. Integraremos modulos, NumPy, funciones, argumentos por defecto y propiedades geometricas.

Ruta del ejercicio

```
01_python_bootcamp/
`-- week_03_modules_and_coordinates/
    `-- class_12_engineering_toolkit/
        `-- exercises/
            `-- ex20_engineering_vector_toolkit/
                |-- main.py
                `-- engineering_toolkit.py
```

Valores de referencia

Variable / caso	Valor	Resultado esperado
v1	[3.0, 4.0]	norma = 5.0
unit(v1)	-	[0.6, 0.8]
v_x	[1.0, 0.0]	angulo(v1,v_x) = 53.1301 deg aprox.
proyeccion v1 sobre x	-	[3.0, 0.0]
rotate [100,0]	90 deg	[0.0, 100.0] aprox.
R(30 deg)	-	det(R)=1 y R.T@R=I

BLOCK A - vector_norm()

```
def vector_norm(vector):
    return np.linalg.norm(vector)
```

Prueba con v1 = np.array([3.0, 4.0]). El resultado esperado es 5.0.

BLOCK B - unit_vector()

```
def unit_vector(vector):
    norm = vector_norm(vector)

    if np.isclose(norm, 0.0):
        return None

    return vector / norm
```

La funcion recibe un vector y devuelve otro con la misma direccion pero norma 1. El vector cero es un caso especial porque no puede normalizarse.

BLOCK C - angle_between_vectors()

```
def angle_between_vectors(v1, v2, degrees=True):
    denominator = vector_norm(v1) * vector_norm(v2)

    if np.isclose(denominator, 0.0):
        return None

    cos_theta = np.dot(v1, v2) / denominator
    cos_theta = np.clip(cos_theta, -1.0, 1.0)

    angle = np.arccos(cos_theta)

    if degrees:
        angle = np.rad2deg(angle)
```

```
    return angle
```

El argumento degrees=True permite devolver grados por defecto, pero puede sobrescribirse con degrees=False.

BLOCK D - project_vector()

```
def project_vector(vector, direction):
    denominator = np.dot(direction, direction)

    if np.isclose(denominator, 0.0):
        return None

    scale = np.dot(vector, direction) / denominator
    return scale * direction
```

Con vector=[3,4] y direction=[1,0], la proyeccion esperada es [3,0].

BLOCK E - Reutilizar la rotacion de EX19

```
def rotation_matrix_2d(angle_deg):
    theta = np.deg2rad(angle_deg)
    return np.array([
        [np.cos(theta), -np.sin(theta)],
        [np.sin(theta),  np.cos(theta)]
    ])

def rotate_vector_2d(vector, angle_deg):
    return rotation_matrix_2d(angle_deg) @ vector
```

BLOCK F - check_rotation_matrix()

```
def check_rotation_matrix(R):
    identity = np.eye(R.shape[0])

    orthogonal = np.allclose(R.T @ R, identity)
    determinant = np.linalg.det(R)
    proper_rotation = orthogonal and np.allclose(determinant, 1.0)

    return {
        'orthogonal': orthogonal,
        'determinant': determinant,
        'proper_rotation': proper_rotation
    }
```

Esta funcion integra diccionarios y algebra lineal: devuelve varias comprobaciones con nombres descriptivos.

BLOCK G - main.py

```
from engineering_toolkit import (
    vector_norm,
    unit_vector,
    angle_between_vectors,
    project_vector,
    rotate_vector_2d,
    rotation_matrix_2d,
    check_rotation_matrix
)

def main():
    # Define reference vectors
    # Call toolkit functions
    # Print a compact engineering summary

if __name__ == '__main__':
    main()
```

El objetivo final es que main.py no contenga formulas: solo importa, coordina y presenta.

VALIDACION FINAL

```
Explain in comments:
- What does vector_norm() represent physically/geometrically?
- Why does unit_vector() fail for the zero vector?
```

```
- What does np.dot(v1, v2) represent?
- Why does the angle formula divide by both vector norms?
- Why do we use np.clip() before np.arccos()?
- What does np.arccos() receive and return?
- What is the difference between a vector and its projection onto another direction?
- Why does check_rotation_matrix() test both R.T @ R = I and det(R) = 1?
- Why is returning a dictionary useful in check_rotation_matrix()?
- What advantages do we gain by putting these tools in engineering_toolkit.py?
```

Criterio de cierre

EX20 estara completado cuando puedas usar tu propio modulo para calcular norma, vector unitario, angulo, proyeccion y rotacion, y validar una matriz de rotacion sin duplicar formulas en main.py.